

Department of Electrical and Computer Engineering

The University of Texas at Austin

ECE460N, Spring 2026

Lab Assignment 5

Code + Test cases Due: Monday Apr 27, 11:59 pm

Documentation Due: Wednesday Apr 29, 11:59 pm REGARDLESS OF IF YOU USE SLACK DAYS OR NOT! We have to have them all in so we can grade them quickly.

- The control store template in [Excel](#) and [PDF](#) formats.
- You can use the following pdf files to show the changes you made to the datapath, state diagram and microsequencer. Please note that you can submit hand drawn diagrams.
 - datapath [pdf file](#)
 - state machine [pdf file](#)
 - microsequencer [pdf file](#)

Make sure your Lab 4 implementation is correct before starting Lab 5. **You have unlimited attempts for to regrade Lab 4.** You may use the `submit-ee360n -Lab4 -grade` command on grader2 to test and debug your Lab 4 program right after you have graded for the first time. See the submission instruction for more details.

In order to encourage you to fix your Lab 4, we will use the most-recently graded Lab 4 grade as a 40% component of your final Lab 4 grade. The other 60% component is your initial lab 4 grade. **Take advantage of this policy to improve your lab 4 grade!**

Note: **Passing all the tests does not mean that your Lab 4 is correct!** An incorrect program may just happen to behave correctly on our test cases. It is your responsibility to ensure that your Lab 4 is correct.

You are expected to finish the design using up to 64 states. If you choose to use more than 64 states, change the shell code by modifying the `CONTROL_STORE_ROWS` macro and any other piece of code related to the state number (e.g. J bits).

If you finish the design using no more than 64 states, we will add 10 points as bonus to your final Lab 5 grade (It is possible to get up to 110 points on your test/grade runs)

Shell Code

A modified shell code has been written for you: [lc3bsim5.c](#)

You will need to copy and paste the code you wrote for Lab 4 into this new shell code. The shell code takes a new command line parameter that specifies the file containing the page table to be loaded into physical memory. To run the simulator, type:

```
lc3bsim5 <micro_code_file> <page_table_file> <user_program>  
<program_file_1> <program_file_2> ...
```

The first parameter is the microcode file as before. The second parameter is the page table in LC-3b machine language. Note that since the page table is in physical memory, the first line of this file should be a physical address. For all the program files (including the interrupt and exception handlers), which are in virtual memory, the first line should be a virtual address.

Introduction

The goal of this lab assignment is to extend the LC-3b simulator you wrote in Lab 4 to handle virtual memory. You will augment the existing LC-3b microarchitecture in order to support virtual to physical address translation. You will also provide microarchitectural support for page fault exceptions and change the protection exception from Lab 4 to be based on access mode in the PTE.

Specification

Virtual memory

The virtual address space of the LC-3b is divided into pages of size 512 bytes. The LC-3b virtual address space has 128 pages, while physical memory has 32 frames. The LC-3b translates virtual addresses to physical addresses using a one-level translation scheme. Virtual pages 0–23 comprise the system space. They are mapped directly to frames 0–23 and are always resident in physical memory. The system space may be accessed with any instruction in supervisor mode, but only with a TRAP instruction in user mode. The remaining virtual pages (24–127) comprise the user space and are mapped to frames 24–31 via a page table stored in system space.

The page table contains PTEs for *both* the system and user space pages. It resides at the beginning of frame 8 of physical memory. A page table entry (PTE) contains only 9 bits of information but, for convenience, is represented by a full 16 bit word. Thus one PTE occupies two memory locations. The format of each PTE is as follows:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	PFN					0	0	0	0	0	P	V	M	R

If the protection (P) bit is set to 0, the page is protected: it can only be accessed in supervisor mode or by a TRAP instruction. Otherwise (P=1), the page can be accessed in either user or supervisor mode. The valid (V) bit indicates whether the page is mapped to a frame in physical memory (V = 1) or not (V = 0). The modified (M) bit indicates whether the page has been written to since it was brought in (M = 1) or not (M = 0). The reference (R) bit is set on every access to the page and cleared every timer interrupt.

Note that the PFN, the modified bit, and the reference bit are meaningless if the page is not resident in memory (the valid bit is zero).

Exceptions

Lab 5 supports four exceptions, listed below in order of priority (highest to lowest):

Exception	Vector	Occurs when ...
Unaligned access	x03	a word-sized memory access to an odd virtual address is attempted
Protection	x04	the machine is in user mode and a protected page is accessed by any instruction other than the TRAP instruction
Page fault	x02	the page accessed by a user program is not valid (not in physical memory)
Unknown Opcode	x05	the program tries to use an unknown opcode (1010 or 1011)

Unaligned access exceptions should be detected when the virtual address is formed. Protection and page fault exceptions should be detected when the machine attempts to create the physical address. Exceptions should be taken as soon as they are detected. Given that a protection exception and a page fault exception are detected at the same time, we take the protection exception. (Do you see why we take the protection exception rather than the page fault?)

Implementation

Additions to Datapath

During the execution of instructions, your microcode will convert virtual addresses to physical addresses and modify PTEs when necessary. To make address translation possible, we have added two registers to the datapath: Page Table Base Register (PTBR) and Virtual Address Register (VA). The PTBR points to the first entry of the page table in physical memory and will be initialized by the shell code to the starting address of the page table file you provide as a command line parameter to the simulator. To put the PTBR register onto the bus, you should assert the GatePTBR signal. The VA register is a temporary register to hold the current address being translated. To put the VA register onto the bus and to write to the VA register from the bus, you should assert the GateVA and LD.VA control signals, respectively.

Address Translation

We will use a simple one-level translation scheme to translate virtual addresses to physical addresses. The high 7 bits of the virtual address (bits[15:9]) specify the page number, and the low 9 bits of the virtual address (bits[8:0]) specify the offset within the page. All 7 bits of the page number will be used during translation. Assume that at the beginning of each address translation phase the virtual address is located in the MAR, and if the operation is a write, a source register holds the data to be written. Address translation consists of the following steps:

1. Save the MAR into the temporary register VA (also check unaligned)
2. Load the MAR with the address of the PTE of the page containing the VA:
 - $MAR[15:8] \leftarrow PTBR[15:8]$

- $MAR[7:0] \leftarrow LSHF(VA[15:9], 1)$
- i.e., $MAR \leftarrow PTBR + (2 \times \text{page_number})$.
- 3. Read the PTE of the page containing the VA into the MDR
- 4. Check for a protection exception
- 5. Check for a page fault
- 6. Set the reference bit of the PTE
- 7. If the pending access is a write, set the modified bit of the PTE
- 8. Write the PTE back to memory
- 9. Load the physical address into the MAR:
 - $MAR[13:9] \leftarrow PFN$
 - $MAR[8:0] \leftarrow VA[8:0]$

To add support for virtual memory, you first need to determine when you need to perform address translation. Then, you will need to determine how to add an address translation state sequence to the state diagram. You will also need to determine how to return back to the correct state once address translation is complete. For this, you will need to augment the microsequencer. You are free to add new control signals, gates, muxes, temporary registers as you wish as long as you fully document your changes.

Writing Code

You will modify your source code and control store from Lab 4 to implement the specification above. **Do not remove Lab 4 functionality, even if it's not mentioned in the specification (for example, the timer interrupt and nested interrupt/exception).** You will also create the following files to construct a particular execution scenario:

- Page table (starting at physical address $\times 1000$). The page table will be initialized upon simulator start-up based on the contents of the page table file you provided as a command line parameter to the simulator. Map pages 0-23 (system space) to frames 0-23 and mark them as valid and protected. Map pages 24-127 (user space) as unprotected. Map pages 24, 96, and 126 (the user stack) to frames 25, 28, and 29 and mark them as valid. Mark all other pages of user space invalid.
- Other files are loaded into virtual memory and are described in the table below:

File	StartingVA	Description
User program	$\times 3000$	This program should calculate the sum of the first 20 bytes stored in memory starting at address $\times C000$. This sum should then be stored as a 16-bit word at $\times C014$. Then the program should jump to the address given by the value of the sum.

User program data	x000	Use the following data: x12, x11, x39, x23, x02, xF6, x12, x23, x56, x89, xBC, xEF, x00, x01, x02, x03, x04, x05, x06, x07
Interrupt/Exception vector table	x0200	Form the contents of this table based on the vectors of each interrupt/exception and starting addresses of the service routines for each interrupt/exception.
Timer interrupt service routine	x1200	The interrupt service routine must traverse the entire page table, clearing the reference bits of each PTE. You may assume when writing this code that the start address of the page table is fixed.
Page fault exception handler	x1400	All four should simply halt the machine
Protection exception handler	x1600	
Unaligned access exception handler	x1A00	
Unknown opcode exception handler	x1C00	

Note: you should also test your design with other execution scenarios.

What to Submit

The following table lists the files you need to submit:

File Name	Description
Readme (20 pts)	A detailed explanation of your design. It should include: Changes you made to the state diagram. Include a picture similar to the state machine that shows the new states you

	added (only show your changes or mark your changes in a new state diagram). This picture should include the encodings of new states you added. Clearly show where each state fits in the current state diagram. Describe what happens in each new state. • How many states in total is used (including those for lab3 and lab4) • Changes you made to the datapath. Clearly show the new structures added, along with the control signals controlling those structures. Describe the purpose of each structure. • New control signals you added to each microinstruction. Briefly explain what each control signal is used for. • Changes you made to the microsequencer. Draw a logic diagram of your new microsequencer and describe why you made the changes. • For microsequencer, please draw the whole thing including lab 4 changes. For the rest, only include changes made in lab 5. Readme is due on November 19th 11:59PM, or one day after your slack day submission. SUBMIT ON GRADESCOPE, AND PLEASE <u>CORRECTLY</u> ASSIGN PROBLEMS TO PAGES OR FACE PENALTY ON EACH PART OF YOUR README. Each part of the readme missing/not assigned could result in up to 5 pts lost.
<code>lc3bsim5.c</code>	Adequately documented source code of your simulator
<code>ucode5</code>	Control store file used by your simulator
<code>dumpsim</code>	To generate this file, set up the execution scenario described above. Run the user program and dump the memory locations containing the page table once before the 300th cycle, once after the ISR is done, and finally after the protection exception halts the execution of the program (you should get a protection exception after the jump). Also, dump memory location $\times 3814$ (corresponding to which virtual address?) and the current registers after the protection exception.
<code>pagetable.asm</code>	Page table
<code>vector_table.asm</code>	Interrupt and exception vector table
<code>add.asm</code>	User program
<code>data.asm</code>	Data for the user program
<code>int.asm</code>	Timer interrupt service routine

Note: all asm files must be LC3b assembly source code files, NOT hex or obj files.

Things To Consider

1. The supervisor stack should start at address `x2FFE` (SSP is initialized in the shell code to `x3000`). If the user program uses the user stack, the user stack should start at address `xFDFE` (i.e. the user program should initialize R6 to `xFE00`).
2. When do you need to do address translation? How will you return from the address translation to the correct state? An obvious solution would be to have a register which holds the 6-bit return address. Although this is acceptable, think about the possibility of other solutions, e.g. a mechanism that is similar to COND bits in the microsequencer?
3. The interrupt service routine needs to access the page table in order to clear the reference bits of each PTE. The page table is in physical memory, and the PTBR contains a physical address. However, the instructions in the interrupt service routine can only access virtual addresses. You may assume that virtual pages 0–23 are always resident and will always be mapped directly to frames 0–23. The page table resides at the beginning of frame 8 of physical memory.
4. The microcode used for starting the exception service routine is very similar to the microcode that is used for starting the interrupt service routine. Are there any differences? With these differences in mind perhaps you can combine the states used for interrupt and exception handling.
5. You do not need to worry about nested interrupts or exceptions for this assignment. However, don't remove the support for nested exceptions/interrupts from lab4
6. If you were unable to finish Lab 4 and are hence starting from Lab 3, please make a note of this in the readme file. You will be unable to simulate the page fault exception, protection exception and timer interrupt if the requirements of Lab 4 are not done.
7. The timer interrupt service routine clears all the reference bits. Note, however, that at the end of this service routine, not all reference bits are 0. Do you see why?